NORTHERN UNIVERSITY OF BUSINESS AND TECHNOLOGY, KHULNA
Department of Computer Science and Engineering

WEEK 02

Project Proposal

CSE 4104 · Software Development III · Section 7C · Fall 2026

SUBMISSION SUMMARY

Official Team Name	CSE4104-7C-T09
Section	7C
Project Title	UniBuddy: An AI Study Companion That Answers Questions From a Student's Own Course Material
Team Leader	Md Nafew Yeasin (11230321355)
Team Size	4 members
Submitted To	Assistant Professor Md. Riaz Mahmud
Submission Date	9 November 2026
GitHub Repository	github.com/hey-naf/cse4104-7c-t09-unibuddy

1 PROJECT TITLE

"UniBuddy: An AI Study Companion That Answers Questions From a Student's Own Course Material"

In one line: *UniBuddy turns a student's own lecture PDFs into a tutor that answers questions about them and cites the page the answer came from, with the student's courses and deadlines tracked alongside.*

The title names the two halves of the system and the relationship between them. UniBuddy is a companion, not a replacement for study: it does not teach a syllabus, it answers questions about the material the student was already given. The second clause is the load-bearing one. A student's own course material is a closed, private, badly formatted set of documents that no public model has read, and answering from it is the one capability that conventional software cannot provide.

2 PROBLEM STATEMENT

2.1 The problem as it currently exists

An undergraduate in Bangladesh carries five or six courses in a single semester. Each course has its own handout, its own deadline pattern, and its own examination date. That information does not live in one place. It arrives as printed sheets distributed in class, as photographs of a whiteboard, as PDFs dropped into a Messenger group at eleven at night, and as whatever the student managed to write down before the lecture moved on. Nothing is indexed. Nothing is searchable. Nothing is guaranteed to still exist when the student needs it in week twelve.

The second half of the problem is worse than the first, and it arrives at examination time. The student now holds several hundred pages of lecture PDFs across five courses and needs to find the specific paragraph that answers the specific question they are stuck on. There is no efficient way to do this. Re-reading everything is too slow, so the student either revises unevenly, guided by whatever feels familiar, or gives up and asks a classmate who is equally unsure. The information is present and inaccessible at the same time.

2.2 Why the problem is important

The cost is paid twice. First in time: hours spent locating material that already exists, rather than understanding it. Second, and more seriously, in marks. Work is missed not because it was difficult but because it was never seen; questions go unanswered not because the answer was absent from the handout but because the student could not find it in four hundred pages.

There is a third cost that is easy to overlook. When a student gives up on the handout and asks a general-purpose chatbot instead, the chatbot has never read that handout. It answers fluently, confidently, and wrongly. The student has no way to tell the difference, because the wrong answer is written in the same register as the right one. A confident error is worse than no answer at all: it is harder to detect, and it is believed.

2.3 Why existing solutions are insufficient

Several mature products address part of this problem. Each was reviewed before committing to the idea. None of them addresses the whole of it.

System	What it already does	Why it is not enough here
Notion / Notion AI	A strong organizer with summarisation and a mature editor.	A general workspace, not course-aware. Its AI does not ground answers in the student's uploaded PDFs with page citations, and the setup is heavy for someone who only wants to see this week's deadlines.
Google Classroom	The institution's own feed of assignments and materials.	Read-only for students. No personal task view, no cross-course summary, and no study assistance of any kind.
ChatGPT / Gemini (general chat)	Strong reasoning and explanation quality.	Has never seen the student's course pack. Answers confidently and without a source, which is the failure mode described in 2.2.
Quizlet	Effective flashcards and practice tests.	Cards must be authored by hand. No document upload, so no question answering over lecture notes.
Anki	Spaced repetition that genuinely improves retention.	Manual card authoring and a steep setup curve. No AI assistance and no document ingestion.
Perplexity	Answers with citations drawn from the live web.	Cites the open web, not the student's closed course material. Cannot answer what a particular lecturer said about a topic.
Studocu / Course Hero	Large libraries of notes shared by other students.	Other people's notes, paywalled, and not guaranteed to match the syllabus being examined.

The pattern in that table is consistent. Each product solves one slice of the problem well, and the slices do not meet. A student who wants deadlines, searchable course documents, and grounded answers currently installs three tools, keeps them in sync by hand, and pays for the integration in attention. That integration cost is the actual gap, and it is why a student ends up with nothing rather than with three partial solutions.

2.4 Why AI is relevant to solving it

Three requirements of this system cannot be met by conventional software, and they are the reason the project is not simply a task manager with a search box.

- **Searching by meaning.** A student's question rarely shares vocabulary with the passage that answers it. Asked why TCP needs a three-way handshake, a keyword search over a networking handout returns nothing useful, because the passage explains that both hosts must agree on initial sequence numbers. The words do not overlap; the meaning does. Matching meaning requires semantic embeddings, not string comparison.
- **Answering from a closed, private document set.** A general model has not read the student's handout. Retrieval-augmented generation restricts the answer to passages retrieved from the student's own uploads, which is what makes a citation possible in the first place. The model's task narrows from knowing everything to reading these paragraphs and explaining them.
- **Explaining at the level the student needs.** The same passage must be legible to a first-year student and useful to a final-year student. Producing that explanation, adapted to the question actually asked, is a language task with no rule-based equivalent.

The organizer half of UniBuddy requires none of this. Courses, deadlines, and a dashboard are ordinary CRUD operations. It exists to give the tutor context, because knowing an examination is three days away changes which questions are worth asking. But it is not where the intelligence lives, and the design says so plainly. If the AI were removed, UniBuddy would be a to-do list and would not be worth building.

3 OBJECTIVES

The objectives below are written so that each one can be checked at the end of the semester rather than merely asserted. An objective that cannot fail is not an objective. Where a target number appears, it is the number the team will report against in the final evaluation, whatever the result turns out to be.

#	Objective	How it will be verified
O1	Answer a question from an uploaded lecture PDF, with the source page cited, in under ten seconds.	Ten fixed questions per test document, each answer checked by hand against the page it cites. Target: at least eight of ten correct and correctly cited.
O2	Decline to answer when the material does not contain the answer, rather than answering from general knowledge.	Twelve out-of-scope questions per document. Target: at least ten refused or explicitly flagged as not found, and none answered confidently and wrongly.
O3	Show that semantic retrieval finds answers that keyword search over the same documents misses.	The same question set run through a keyword baseline and through the embedding pipeline. Both hit rates reported side by side.
O4	Let a student record each course and assessment once, and see everything due in the next seven days on a single screen.	A walkthrough with a full semester of real course data, timed end to end. No item may be entered twice.
O5	Deliver a publicly reachable system with real accounts and strict per-user isolation.	Two test accounts. Cross-account reads of documents, questions, and answers attempted directly against the API. All must fail.

Two of these carry more weight than the rest. O1 and O5 are the objectives that cannot be dropped if the schedule slips, because without them there is no product to demonstrate, only a document viewer with a login screen. O2 and O3 are what separate a grounded tutor from a chat window pointed at a PDF, and they are the evidence the AI component is doing real work. O4 is the supporting scope: valuable, but the first thing to narrow if time runs short.

4 PROPOSED FEATURES

Features are listed by area and marked for the first working release. The split is deliberate: the left column is what makes the product exist, the right column is what makes it complete. Only the left column is promised for this semester.

Area	Feature	What it does	V1
Account	Registration and login	Email and password, hashed with bcrypt, JWT session. All data private to the account.	Yes
Account	Student profile	University, department, and semester, so explanations can be pitched at the right level.	Yes
Organizer	Course management	Add, edit, and remove the courses enrolled this semester.	Yes
Organizer	Assessment tracking	Assignments, quizzes, and examinations with course, due date, and status.	Yes
Organizer	Dashboard	Everything due in the next seven days, grouped by course.	Yes
Tutor	Document upload	Lecture PDFs uploaded and filed under the course they belong to.	Yes

Area	Feature	What it does	V1
Tutor	Indexing pipeline	Text extracted, split into overlapping chunks, embedded, and stored in pgvector alongside the page number.	Yes
Tutor	Grounded question answering	A plain-language question answered only from retrieved chunks, with the page cited.	Yes
Tutor	Source display	The cited page shown beside the answer so the student can check it against the handout.	Yes
Platform	Responsive interface	Usable on a phone between classes and on a laptop while studying.	Yes
Platform	Persistent storage	All data held in PostgreSQL and restored on next login.	Yes
Tutor	Quiz generation	Practice questions generated from uploaded material.	Later
Organizer	GPA and attendance	Semester grade records, CGPA calculation, attendance tracking.	Later
Tutor	Bangla explanations	Answers and explanations in Bangla as well as English.	Later
Helpdesk	Institutional Q&A	Questions over handbooks, syllabi, and notices.	Later
Collaboration	Group projects	Shared projects, task assignment, and progress tracking.	Later

The cut falls after the source display for a specific reason. Up to that point every feature is load-bearing for one demonstrable flow: log in, upload a handout, ask a question, receive an answer with a page number that can be verified against the original. That single flow exercises authentication, the database, file storage, the backend, the frontend, the AI pipeline, and the deployment. Everything after it is an addition to a product that already works, which is the correct order to build in.

5 TECHNOLOGY STACK

The stack follows the course handbook's recommendations and is chosen so that four people can build, test, and deploy it within one semester. These are initial selections and may be revised after the system design in Week 4.

Layer	Selection	Rationale
Frontend	React 18 with Vite, styled with Tailwind CSS	Component structure suits a dashboard; Vite keeps the build fast; Tailwind gives a responsive layout without a hand-written stylesheet.
Backend	Node.js with Express	Same language as the frontend, so the team does not context-switch. The handbook's recommended backend, with direct access to AI provider SDKs.
Database	PostgreSQL, hosted on Supabase	Relational model fits users, courses, assessments, and documents. The pgvector extension stores embeddings in the same database, avoiding a second service to operate.
AI	Google Gemini API, chat and embeddings	Multimodal, with a usable free tier for development and a generous context window. OpenAI is kept as a fallback behind a single provider interface.
Retrieval	Text chunking with pgvector similarity search	Grounds every answer in the student's own material and returns the page number the citation needs.
Authentication	JSON Web Tokens with bcrypt password hashing	Named explicitly in the handbook (Section 80.1). Keeps sessions stateless and passwords unrecoverable.
File storage	Supabase Storage, private bucket	Uploaded handouts are personal. Per-user access policies prevent one student reading another's files.
Deployment	Vercel, Render, and Supabase	Free tiers are sufficient for a semester project, and all three accept secrets through environment variables.
Version control	Git with GitHub	Mandatory for the course. Branch strategy and commit conventions are documented in the repository README.

The stack is also defined by what was ruled out. Flutter was rejected because a mobile build pipeline earns no additional marks and doubles the surface the team must test. Django and Laravel were rejected because splitting the team across two languages slows integration without improving the result. A dedicated vector database was rejected because pgvector handles a corpus of this size inside the database already in use. Each of these is a defensible choice in a different context; none of them is defensible for four students with fourteen weeks.

6 EXPECTED OUTCOME

6.1 What the student gains

A student who uses UniBuddy ends the semester with one place that holds their courses, their deadlines, and their course documents, and one interface that can answer questions about those documents with a reference back to the page. The gain is not that the student studies less. It is that the time spent studying goes into understanding rather than into locating. The difference between hunting through four hundred pages and reading the right paragraph is the whole of the benefit.

6.2 How the existing workflow changes

Stage	Today	With UniBuddy
Collecting material	Handouts arrive as printouts, photographs, and files in a group chat. Filing is manual and inconsistent.	Each handout is uploaded once, under the course it belongs to, and stays retrievable.
Tracking deadlines	Dates live in memory, in a paper diary, and in scattered chat messages. Some are missed without ever being seen.	One dashboard lists everything due in the next seven days, across every course.
Revising for an exam	Re-read the whole document, or guess at which sections matter, or ask a classmate who is equally unsure.	Ask a specific question, receive a specific answer, and check it against the cited page.
Checking an answer	Ask a general chatbot, receive a fluent answer with no source, and have no way to verify it.	Every answer carries a page reference into the student's own handout, so verification takes seconds.

6.3 What the AI integration specifically contributes

It is worth being precise here, because the course rejects AI that is bolted on rather than depended upon. The AI in UniBuddy contributes three things that would be absent from a version of the same application built with ordinary code.

- **Retrieval that works on real questions.** Keyword search over the same documents returns almost nothing for a naturally phrased question. Semantic retrieval is the only mechanism that connects the question to the passage, and without it the document store is a file cabinet with a broken index.
- **An explanation rather than an excerpt.** A retrieved chunk is not an answer. Turning it into an explanation pitched at the level of the person asking is a language task, and it is the difference between handing a student a page number and teaching them something.
- **Refusal as a feature.** Constraining the model to retrieved passages, and requiring it to say when nothing relevant was retrieved, is what makes the output trustworthy enough to rely on before an examination. A system that always answers is useless precisely because it always answers.

The measurable outcome for the semester is a deployed application in which a student can complete that flow unaided, and a written evaluation reporting the figures from objectives O1, O2, and O3. Those figures, reported honestly even where they fall short, are the evidence that the AI is doing real work rather than decorating a CRUD application.

7 INITIAL PROJECT PLANNING

The plan below maps the fourteen teaching weeks onto deliverables that can be shown, not activities that can be claimed. Each phase names the member who owns it and the artefact that will prove it was done.

7.1 Phase plan

Weeks	Phase focus	Deliverable and evidence	Owner
1-2	Definition	Team formed, idea selected, repository live, proposal and team sheet submitted.	Naf
3	Requirements	SRS with functional and non-functional requirements, user roles, and use cases, plus the initial diagrams.	Naf, all four
4	System design	System architecture diagram, ER diagram, use case diagram, and the API contract that every later week builds against.	Russell, Naf
5	Interface design	Figma wireframes for login, dashboard, tutor, and profile screens, UI screenshots, and the development roadmap.	Abid
6	Backend foundation	Database schema migrated, authentication working, CRUD endpoints for courses and assessments, documented in a collection.	Russell
7	Frontend application	React application running against the live API. Every V1 screen reachable and wired.	Abid
8	AI pipeline	PDF extraction, chunking, embedding, retrieval, and cited answering. Evaluation harness producing the O1, O2, and O3 figures.	Fahad
9	Integration and completion	All V1 features working end to end. Every item on the course's minimum requirements checklist demonstrable.	All four
10	Testing	Test plan, executed test cases, and a bug log with resolutions. Includes AI answer quality, not only interface behaviour.	All four
11	Deployment	Frontend, backend, and database live on public URLs, secrets in environment variables, no credentials in the repository.	Russell
12	Documentation	Final report, user manual, and API documentation, with screenshots of the deployed system.	Naf
13	Presentation preparation	Slide deck and a rehearsed demonstration, including a fallback recording in case the live deployment fails.	All four
14	Final presentation and viva	Delivered demonstration. Each member able to explain their own module and the design decisions behind it.	All four

7.2 Responsibilities

Roles follow the handbook's recommendation (Section 35.2). Every member owns one technical area and is expected to be able to defend it individually, because individual contribution is marked separately from team performance.

#	Member	Student ID	Responsibility
1	Md Nafew Yeasin	11230321355	Team leader. Coordination, weekly submissions, requirements and design documents, final report.
2	Mujibur Rahman Fahad	11230321568	AI integration lead. Extraction, chunking, embeddings, retrieval, prompt design, and the evaluation of answer quality.

#	Member	Student ID	Responsibility
3	Abid Hasan	11230321553	Frontend developer. React application, responsive layout, and the interface between the tutor and the student.
4	M R Russell	11230321545	Backend developer. API, database schema, authentication, file storage, and deployment.

7.3 Risks and mitigations

Risk	Effect if it happens	Mitigation
Scope exceeds the semester	Several half-built modules and a weak demonstration.	V1 is fixed to the single flow in Section 4. Later features are added only after that flow is reliable.
The AI becomes a thin API wrapper	Loses most of the fifteen marks allocated to AI integration.	Answers are grounded in retrieved chunks, citations appear in the interface, and retrieval is logged and evaluated rather than assumed.
Free-tier quota is exhausted	The demonstration fails at the worst possible moment.	Cache repeated queries, cap tokens per request, and record a working demonstration before the viva.
Scanned PDFs defeat extraction	Retrieval returns noise and answers become wrong.	V1 accepts text-based PDFs only. Scanned files are detected and rejected with a clear message rather than silently indexed.
A member becomes inactive	Reduced individual marks and an uneven commit history.	One module per member from Week 6, weekly commits, and a team agreement on escalation if someone misses two consecutive weeks.
Weekly submissions slip	Ten marks of continuous assessment erode and cannot be recovered.	Something is submitted every week, however small. A partial document scores better than an empty slot.

7.4 Repository and working practice

The repository is public and holds the project from the first week rather than being assembled at the end. From Week 3 onward, work lands on a feature branch, merges into development after review, and reaches main only when it is deployable. Branch names follow the course handbook: main, development, frontend, backend, and ai-integration. Commit messages describe what changed, because commit quality is marked directly.

Documentation is generated from source rather than written by hand into a PDF, so that the team roster and project details cannot drift between the proposal, the team information sheet, and the submission cover. Correcting a name or a student ID in one place regenerates every document at once, which is why the four names and IDs match in all three.

8 DECLARATION

We confirm that this document describes our own original project idea. The systems listed in Section 2.3 were reviewed for comparison only; no code, content, or text has been copied from them. Every member of the team has read this proposal and can explain the technical decisions it contains.

Name	Student ID	Signature
Md Nafew Yeasin	11230321355	
Mujibur Rahman Fahad	11230321568	
Abid Hasan	11230321553	
M R Russell	11230321545	